USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

The ADC, and a Wrong Bit in a Right Register

UM-NATOS-024

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-024	1.0 · 2026-08-16	**Verified on hardware**	Internal

1. Abstract

Every driver before this one talked to a device that answers in protocol. A display either shows the pattern or it does not; a flash read either returns the magic number or it does not; a touch controller either responds on the bus or is silent. Each of those has an internal check.

This one returns a number meaning "how much light", and nothing in the system can check it for plausibility. 619 is a perfectly good ADC reading. So is 351. So is the same value on all eight channels, which is what a converter that is not converting anything returns.

That is what shapes this report: the driver is ordinary, and the verification is the work.

2. ADC1, and a constraint that became an advantage

ADC2 is unusable on this part whenever WiFi is running, which is the standard reason to avoid it. **That reason does not apply here.** The `-mabi=call0` build cannot link Espressif's precompiled radio libraries at all (UM-NATOS-003 §5.1), so WiFi will never run and ADC2 is permanently free.

ADC1 is still first, because the board's own light sensor is on it and a sensor already soldered down needs no wiring to be wrong about.

3. The defect was one bit

`SENS_SAR1_EN_PAD_FORCE` is `BIT(31)`. It was written as bit 27, from memory.

Bit 27 is not a spare bit. `SAR1_EN_PAD` is **twelve** bits at shift 19, so 27 sits inside it: setting it selected a channel that does not exist, and left pad selection with the hardware FSM. The FSM then measured whatever it liked, identically, forever.

The symptom was a complete set of reassurances:

observation	what it seemed to prove
all 8 channels ≈ 619	plausible mid-scale readings
`DONE` asserted every time	conversions completing
0 timeouts	the converter is alive
every register read back as written	the configuration is correct

A wrong bit in the right register is invisible to a read-back. That is the one failure mode the read-back discipline of UM-NATOS-023 cannot catch, and it is worth naming because that discipline had been working well enough to feel sufficient.

4. What finally worked was reading the source of truth

Three probes were built before the answer arrived, and each is kept because each answers a question that would otherwise be asked again:

- `adcconv` — proves a conversion *actually runs*. `DONE` goes low when START is cleared and high 13 spins later. This killed the plausible theory that the poll was returning stale latched data.
- `adcdrive` — sweeps against **GPIO32, a pin this kernel drives**. Every earlier probe depended on GPIO36 idling high, which is a belief about the board; this commands the input voltage instead. It also settled `DATA_INV` empirically: driving the pin high *raises* the reading, so the inversion as configured is correct.
- `ldrscan` — watches all eight channels at once.

None of them found it. What found it was **fetching the vendor's `sens_reg.h` instead of recalling it**, at which point every other offset and field in the file turned out to be correct. It was one line.

> This was the third register-layout error in a day (UM-NATOS-023 §5.1, §5.4) and the first fixed by reading the definitions rather than probing around them. Probing is the right tool when the question is *what does this hardware do*. It is the wrong tool when the question is *what is this constant*, and those are easy to confuse when both present as "it does not work".

5. The diagnostic that hid the answer

The RTC IO dump skipped registers reading zero, to make the block's shape legible. The two registers that mattered — the pad muxes at `+0x7c` and `+0x80` — **were zero precisely because they were the thing not yet configured.**

The filter removed exactly the registers the dump existed to find. A bit sweep then ran against `+0x00`, which is `RTC_GPIO_OUT` and governs nothing here, and reported thirty-two failures that meant nothing at all.

> A diagnostic that suppresses the uninteresting cannot find something whose signature is *being uninteresting*.

It prints zeros now. The block's shape was recoverable anyway: the ten-register run at `+0x94..+0xb8` can only be the ten touch pads, which fixes every other offset in the block by counting.

6. Verification

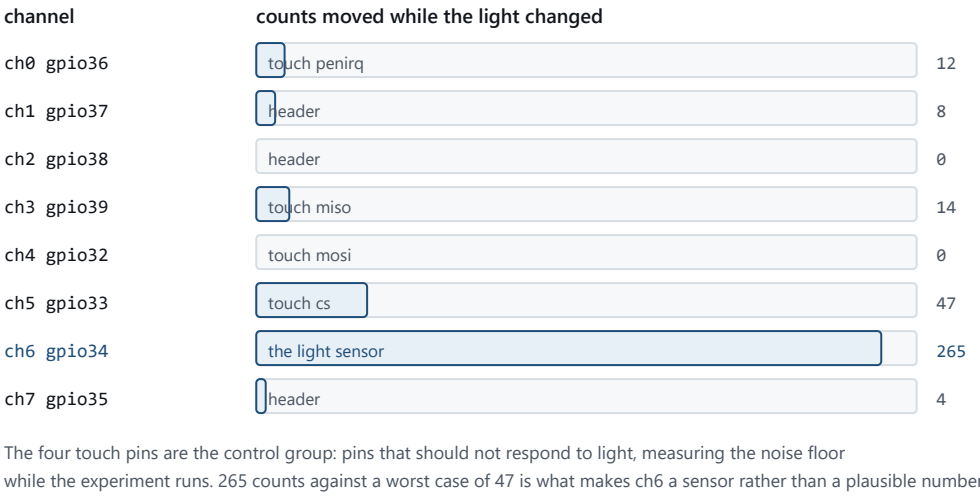

Figure 1. *Every ADC1 channel watched while a hand passed over the board. The touch pins are the control group.*

A hand passed over the board, all eight channels watched at once:

```
ch  gpio   min   max  spread
 0   36    176   188     12   (touch pin)
 1   37    552   560      8
 2   38      0     0      0
 3   39    226   240     14   (touch pin)
 4   32      0     0      0   (touch pin)
 5   33   1777  1824     47   (touch pin)
 6   34    273   538    265   <- the sensor
 7   35    621   625      4
```

265 counts against a control group that never exceeded 47. The four touch pins are kept in the scan for exactly this reason: they are pins that should not respond to light, so they measure the noise floor while the experiment runs.

This also settled a claim rather than repeating one. "The LDR is on GPIO34" was an assertion about the board, propagated through comments; `ldrscan` tested it. It happened to be right. The point is that it is no longer an assumption.

7. Metrics

Quantity	Value
Channels	8, ADC1
Resolution	12-bit
Attenuation	11 dB, all channels
Conversion time	~13 poll iterations
Light sensor	channel 6, GPIO34, **confirmed**
Sensor swing (hand over board)	265 counts
Control-group maximum	47 counts
Lines of driver	~150
Wrong bits	1

8. What this does not establish

- **The reading is uncalibrated.** Counts, not volts. The ESP32's ADC is markedly non-linear near both rails and every part has factory calibration data in eFuse that this driver does not read.
- **The sensor's usable range is narrow.** 273–538 of a possible 0–4095. Enough for light/dark, not characterised for anything quantitative, and the low end was a hand rather than darkness.
- **It is polled.** Every conversion spins. There is no interrupt and no averaging over time, so continuous sampling costs a task.
- **No applications can reach it.** Like every other peripheral here, this is kernel-only — there is no `SYS ADC` (UM-NATOS-022 §2).
- **Only channel 6 has been proven to track anything.** The other seven return distinct, stable, plausible values, and plausible is exactly what this whole report is about not trusting.
- **ADC2 is untouched**, despite being permanently free here. §2.

9. References

- UM-NATOS-003 §5.1 — the ABI decision that makes ADC2 free
- UM-NATOS-017 §3.2, §3.3 — silence reading as a valid measurement, and the first-conversion defect this driver's discard exists to avoid
- UM-NATOS-023 §5.1, §5.4 — the two earlier recalled-layout failures
- `kernel/adc.c` — the driver and all four probes